ComNetX: Local Hierarchical Adaptation for Batched Dynamic Community Detection

GRIGORIY BOKOV[1], ALEKSANDR KONOVALOV[1], ANNA UPOROVA[1], ALEXANDER DROBYSHEV[1], and IAROSLAV EGOROV[1]

[1]AI Research Center, Lomonosov Moscow State University, 1 Leninskie Gory, Moscow 119991, Russian Federation

Corresponding author: Grigoriy Bokov (e-mail: grigoriy.bokov@math.msu.ru).

This work was supported by the Ministry of Economic Development of the Russian Federation under subsidy agreement 000000C313925P4H0002, Grant 139-15-2025-012.

ABSTRACT Dynamic community detection requires updating a graph partition as edges change, but rerunning a detector after every update batch can be expensive. We present ComNetX, a hierarchical method that adapts existing community detectors to batched edge updates. For each batch, ComNetX expands the set of affected endpoints to include their r-hop neighbors and selects all communities intersecting this neighborhood. It then contracts vertex groups defined by a stored hierarchy into weighted supernodes, applies the detector to the reduced graph, and maps the partition back to the original vertices. The finest partition is reported, while subsequent passes update the coarser hierarchy levels for future batches. This procedure preserves nesting across hierarchy levels. Contraction preserves modularity on the selected induced subgraph for partitions that keep each contracted group intact. We analyze time and memory costs and evaluate update streams constructed from six real graphs. In five maintenance comparisons initialized from approximately 99.9% of each graph's stored update records, ComNetX reduces measured processing time relative to warm-started Leiden on the full graph by factors of 6.60–98.72, with absolute differences in final modularity of 0.0035–0.0213. In 500-update experiments on PubMed and ArxivMath covering the final 10% of each stored record sequence, the speedup factors are 24.46 and 72.67, respectively, while the absolute modularity difference remains below 0.008 at every update. The time advantage is smaller during graph growth from singleton initialization. Repeated paired experiments with a feature-aware backend also show time savings under different cluster-count rules for full and local calls.

INDEX TERMS community detection, dynamic graphs, graph clustering, hierarchical graph contraction, incremental graph analytics

I. INTRODUCTION

COMMUNITY structure helps characterize communication networks [1] and the evolution of social groups [2]. Maintaining a partition as edges change is a central task in dynamic community detection [3]. Reapplying a detector to the full graph after every batch can be expensive even when the changes are local. This cost arises with both modularity heuristics, such as Louvain and Leiden [4], [5], and attributed methods whose calls may include feature construction or iterative training [6]–[8].

A static detector can process each snapshot independently or use the previous partition as an initialization, as in the unseeded and seeded Grappolo schemes [9]. Incremental methods reduce repeated work through modularity optimization, screening of affected vertices, or dynamic frontier expansion [10]–[13]. Their update rules are formulated for a particular objective or optimization procedure. Transferring these rules to another detector requires further algorithmic work. This motivates an update procedure that reuses a detector through its partitioning interface.

ComNetX reuses a detector by changing the graph supplied to it. For each batch, it expands the set of affected endpoints to include their neighbors and selects every current community intersecting that neighborhood. This makes every vertex in the selected communities part of the update region. A stored hierarchy then defines groups of vertices to contract into weighted supernodes, reducing the number of vertices presented to the detector. The returned partition is mapped back to the original graph. The first pass updates the reported finest partition; later passes update the coarser hierarchy levels for future batches. Because contracted groups cannot split and edges leaving the selected region are omitted, the resulting optimization is a local approximation to full recomputation.

The contributions of this work are as follows.

1) We introduce ComNetX, which selects complete communities around each edge-update batch, contracts vertex groups defined by a stored hierarchy, and reuses a compatible detector on the resulting weighted graphs. The same construction accepts topology-only and feature-aware backends, with node attributes ag-

gregated over the contracted groups for the latter.
2) We establish that the fine-to-coarse update preserves a nested partition hierarchy. We apply the standard modularity-preserving contraction identity to the selected induced subgraphs and derive time and memory bounds for the update procedure, including its graph-wide worst case.
3) We evaluate the method on streams constructed from six real graphs, including 500-update maintenance sequences, graph growth, batch-granularity controls, and parameter and feature studies. The comparisons quantify the observed quality–time trade-off against full Leiden recomputation and a native dynamic implementation, and examine a feature-aware adapter.

II. RELATED WORK

A. COMMUNITY DEFINITIONS, OBJECTIVES, AND EVALUATION

Community detection encompasses several notions of a cohesive vertex group [14], [15]. Definitions based on internal and external connectivity [16], [17] coexist with optimization criteria and statistical models. Modularity compares within-community edge weight with a degree-preserving null model [18], [19]; normalized cut measures separation relative to group volume [20]; and the map equation describes communities through random-walk compression [21]. Stochastic blockmodels instead specify edge probabilities between latent vertex groups [22]. These formulations motivate different detectors and different notions of partition quality.

For modularity optimization, Louvain combines local moves with graph aggregation [4], and Leiden adds refinement to address poorly connected communities [5]. Shared-memory heuristics and fast Leiden implementations reduce the cost of processing large static graphs [23], [24]. Other approaches pursue efficiency through fast label propagation [25], modularity-guided label updates [26], greedy modularity disassembly [27], and iterated local moves [28]. Efficient static optimization provides a useful baseline, but selecting which parts of a partition to reconsider after a batch is a separate problem.

Evaluation must also distinguish the optimization criterion from external agreement. Modularity has resolution effects and a highly degenerate optimization landscape [29], [30], while comparative studies show that algorithm rankings depend on graph structure and evaluation design [31]. Class labels can reflect a different organizing principle from the graph's structural communities [32], [33]. We therefore report modularity separately from normalized mutual information (NMI) with reference labels [34].

B. INCREMENTAL AND PARALLEL COMMUNITY DETECTION

Dynamic community detection includes maintaining partitions across successive snapshots and explicitly modeling their evolution [3]. Incremental maintenance reduces repeated work through batch-update rules [35], dynamic label propagation [36], [37], and modularity-based updates [10], [38]. Delta-Screening uses modularity-based tests to select vertices for reconsideration [11]. C-Blondel takes a complementary approach: it represents previously detected communities as supernodes in a compressed graph and applies Louvain to that graph [39].

Local work selection and parallel execution are complementary strategies. Dynamic-frontier methods expand the set of vertices reconsidered by the optimizer [12], [40]. Sattar et al. compare dynamic community-detection strategies and develop the shared-memory DyComPar algorithm [41]; DyG-DPCD uses distributed parallel processing [42]. The broader literature on fully dynamic graph algorithms places such update procedures within the problem of maintaining graph properties under insertions and deletions [43].

For Leiden, Sahu et al. combine affected-vertex processing with selective refinement and changes to community renumbering and aggregation [13]. Subsequent work adds community-identifier tracking and a refinement rule based on accumulated changes in community edge weight [44]. LD-Leiden repairs a stored hierarchy around an affected frontier [45]; HIT-Leiden maintains connected components and hierarchical community structures to restrict the affected range [46]. These methods are especially relevant to ComNetX because they combine locality with information retained across updates, although their update procedures modify the Leiden optimization process itself.

C. TEMPORAL MODELS AND LOCAL OUTPUT REQUIREMENTS

Temporal methods can incorporate more than the current snapshot. NOME balances snapshot quality and temporal smoothness [47], and dynamic stochastic blockmodels describe the evolution of network structure through a state-space model [48]. Flow stability uses a dynamical process that accounts for temporal edge ordering [49]. Matthew-effect models [50] and modularity-based community tracking [51] offer further formulations of community evolution. Continuous-time L-modularity avoids choosing snapshot boundaries [52]; other work addresses sparse temporal observations [53] and random-walk snapshot clustering [54]. These formulations give temporal information a role in defining or tracking communities. ComNetX addresses the computational task of updating a partition after each batch without introducing a temporal objective.

Locality can also refer to the requested output rather than the portion of computation that is updated. Seed-, query-, and anchor-based methods retrieve one or several nearby communities [55]–[57], while OLCPM maintains overlapping communities online [58]. ComNetX instead returns a complete, nonoverlapping partition and retains assignments outside its update region. This distinction is necessary when comparing local search with local maintenance of a full partition.

D. ATTRIBUTED AND REPRESENTATION-LEARNING METHODS

Attributed community detection combines topology with node information [59]. Graph neural networks provide a representation-learning framework for graph-structured inputs [60]. Community-oriented methods include learned detection heuristics in SEAL [61], Deep Graph Infomax in CommDGI [62], and neural modularity methods such as DMoN and DGCluster [6], [63]. MAGI connects modularity maximization with contrastive learning [7]. Comparative and survey studies document the resulting range of representations, clustering losses, and evaluation choices [64], [65].

The choice of learning objective and output constraints matters independently of update efficiency. DAG learns communities without a prescribed cluster count [66]. Deep structural entropy supplies another unsupervised clustering objective [67], while fairness-aware spectral and deep methods incorporate group-sensitive connectivity [68]. Efficiency-oriented approaches include pre-training followed by refinement [69] and inductive graph partitioning [70]. Spectral Subspace Clustering for Attributed Graphs (S^2CAG) provides a feature-aware spectral alternative [8], which we use to evaluate the attributed interface.

Temporal learning and downstream prediction impose additional requirements. Kong et al. combine matrix factorization clustering (MFC) with the topological regularizer TopoReg to encourage consistent inter-community structure across successive snapshots [71]. Community structure also supports dynamic popularity prediction [72]. Such uses illustrate why computational reuse must be distinguished from preserving a particular learned representation or temporal objective. ComNetX supplies a contracted adjacency and aggregated attributes; this interface alone does not establish preservation of a feature-aware objective. Its S^2CAG results are therefore interpreted under the implemented feature-processing and cluster-count rules.

E. RELATION TO COMNETX

ComNetX applies locality and contraction through the graph supplied to an existing detector. It selects complete communities intersecting an expanded neighborhood, contracts groups defined by a stored hierarchy, and lifts the resulting partition to the original vertices. The hierarchy is maintained by ComNetX rather than by the detector. This separates the update construction from the detector's internal optimization procedure, while retaining explicit restrictions on which vertices can be separated within each call.

The adapter does not retain the detector's internal state between quotient calls. A native dynamic solver is compatible only if it can independently partition each supplied graph; DF-Leiden used this way is reinitialized on every quotient and differs from its persistent native streaming mode. Our comparisons distinguish these execution paths rather than treating them as the same dynamic algorithm.

III. PROBLEM FORMULATION

A. BATCHED GRAPH MODEL

Let $G_t = (V, E_t, \boldsymbol{A}_t, \boldsymbol{X})$ be the graph after batch t, with a fixed vertex set V, $n = |V|$, sparse adjacency $\boldsymbol{A}_t$, and optional node-feature matrix $\boldsymbol{X} \in \mathbb{R}^{n\times d}$. A sparse signed batch $\boldsymbol{\Delta}_t$ updates the stored adjacency as

$$\boldsymbol{A}_t = \boldsymbol{A}_{t-1} + \boldsymbol{\Delta}_t. \tag{1}$$

The affected endpoints are

$$S_t = \{i : \exists j,\ \Delta_t(i,j) \neq 0 \text{ or } \Delta_t(j,i) \neq 0\}. \tag{2}$$

The resulting adjacency is required to be nonnegative. The experiments use edge insertions and increments to existing edge weights, with fixed vertex attributes; deletions and weight decrements are not evaluated.

We treat a community-detection backend as a partition-producing procedure

$$\mathcal{B}(\boldsymbol{A}, \boldsymbol{X}, \boldsymbol{y}_0; \boldsymbol{\theta}) \longmapsto \boldsymbol{y}, \tag{3}$$

where $\boldsymbol{X}$ and the initial partition $\boldsymbol{y}_0$ are optional, and $\boldsymbol{\theta}$ denotes backend-specific settings such as resolution or a fixed iteration budget. The interface requires one integer label per input vertex; equality of labels defines the returned partition, while their numerical values and ordering are immaterial. This interface covers modularity heuristics and attributed clustering methods. It also covers a native dynamic implementation when that implementation can be called independently on each supplied graph and returns one label per input vertex. ComNetX does not reuse backend state between its quotient calls; each backend retains its own objective and optimization procedure.

Full-snapshot recomputation at update t invokes $\mathcal{B}$ on $\boldsymbol{A}_t$. When the backend accepts an initialization, its own previous partition is supplied:

$$\boldsymbol{y}_t^{\text{full}} = \mathcal{B}(\boldsymbol{A}_t, \boldsymbol{X}, \boldsymbol{y}_{t-1}^{\text{full}}; \boldsymbol{\theta}). \tag{4}$$

The full Leiden baseline follows this warm-started definition. ComNetX instead supplies the detector with graphs constructed from its stored partition hierarchy, as described in Section IV.

B. PARTITION QUALITY

All main comparisons use an undirected graph representation. Directed source graphs are symmetrized within each paired full/local experiment. For a weighted adjacency, let $W = \sum_{i,j} A_{ij} > 0$, $d_i^{\text{out}} = \sum_j A_{ij}$, and $d_j^{\text{in}} = \sum_i A_{ij}$. For a partition C, we compute directed modularity at resolution γ as

$$Q_\gamma(\boldsymbol{A}, C) = \frac{1}{W} \sum_{i,j} \left(A_{ij} - \gamma \frac{d_i^{\text{out}} d_j^{\text{in}}}{W} \right) \mathbf{1}\{C(i) = C(j)\}, \tag{5}$$

which reduces to the standard undirected expression when $\boldsymbol{A}$ is symmetric and $d_i^{\text{out}} = d_i^{\text{in}} = d_i$. The main experiments use $\gamma = 1$. We also use normalized mutual information (NMI),

$$\text{NMI}(C, Z) = \frac{2I(C;Z)}{H(C) + H(Z)}, \tag{6}$$

where I denotes mutual information, H denotes entropy, and Z contains the reference class labels [34].

IV. COMNETX METHOD

A. UPDATE PROCEDURE AND STORED STATE

Each ComNetX update selects a region of the graph, divides its vertices into groups that will be kept together, and contracts those groups into a weighted graph for the detector. We denote the update region at level ℓ by U_t^ℓ, its partition into contraction groups by $\mathcal{G}_t^\ell$, and the contracted adjacency by $\bar{\boldsymbol{A}}_\ell$. We call these indivisible groups *atoms*. All update regions are selected from the previous hierarchy before any level is changed. The method then processes the levels from fine to coarse, calls the detector once per level, and lifts each returned partition to the original vertices. Fig. 1 summarizes the procedure.

The maintained state consists of the accumulated sparse adjacency $\boldsymbol{A}_t$, the optional feature matrix $\boldsymbol{X}$, and an $L\times n$ array of integer community labels $\boldsymbol{C}_t = (C_t^0,\dots,C_t^{L-1})$, where $C_t^\ell(v)$ is the community label assigned to vertex v at level ℓ after batch t. Let $\mathcal{P}_t^\ell$ denote the partition induced by C_t^ℓ. For any two partitions $\mathcal{P}$ and $\mathcal{Q}$ of the same vertex set, we write $\mathcal{P}\preceq\mathcal{Q}$ if every block of $\mathcal{P}$ is contained in a block of $\mathcal{Q}$; then $\mathcal{P}$ refines $\mathcal{Q}$. We use the implementation order in Fig. 1: C_t^0 is the finest, reported partition and C_t^{L-1} is the coarsest level. Thus every block of $\mathcal{P}_t^\ell$ is contained in a block of $\mathcal{P}_t^{\ell+1}$.

Every stored block H is represented canonically by $\kappa(H)=\min H$. Canonical labels are deterministic original-vertex indices and, unlike raw backend labels, cannot collide for disjoint blocks. In maintenance experiments, a designated full-graph method initializes $\mathcal{P}_0^0$; we use Leiden, including for feature-aware backends. Each successive coarser partition is obtained by partitioning the quotient graph of the preceding level and lifting its labels to V. The initial hierarchy is therefore nested, although adjacent levels need not be distinct. The graph-growth control instead initializes every hierarchy level with singleton blocks. These are the partitions maintained by ComNetX, rather than the internal levels of a backend run. Only C_t^0 is reported; the coarser partitions determine contraction groups in later batches.

B. SELECTING THE UPDATE REGIONS

After applying a batch, ComNetX expands S_t by r hops on the support of $\boldsymbol{A}_t+\boldsymbol{A}_t^{\mathsf{T}}$. Nonnegativity ensures that the sparse matrix–vector traversal computes reachability on this symmetrized support. Let

$$B_t = N_r(S_t) = \{v : \operatorname{dist}(v,S_t)\le r\}. \tag{7}$$

Before any partition is modified, an update region is computed at each level from the previous hierarchy:

$$U_t^\ell = \bigcup_{\substack{H\in\mathcal{P}_{t-1}^\ell\\ H\cap B_t\neq\varnothing}} H, \qquad 0\le\ell<L. \tag{8}$$

This closure is the smallest union of complete $\mathcal{P}_{t-1}^\ell$-blocks containing B_t. Because the stored partitions are nested, the corresponding update regions are nested as well,

$$B_t\subseteq U_t^0\subseteq U_t^1\subseteq\cdots\subseteq U_t^{L-1}. \tag{9}$$

A fine block meeting B_t lies in a coarse block that also meets B_t.

C. CONTRACTION AND PARTITION UPDATE

1) Atoms and Quotient Construction

The first pass, at the reported level 0, uses the base atoms

$$\begin{aligned}\mathcal{G}_t^0 = {}&\{\{v\} : v\in B_t\}\\ &\cup\big\{(H\cap U_t^0)\setminus B_t : H\in\mathcal{P}_{t-1}^{L-1},\\ &\qquad (H\cap U_t^0)\setminus B_t\neq\varnothing\big\}.\end{aligned} \tag{10}$$

During the first pass, vertices in B_t can be reassigned individually. The remaining vertices in U_t^0 are kept together according to their coarsest pre-update community. Such a group may contain vertices from several communities at the finest level. Thus the update region retains the full membership of every selected community, while contraction restricts how those vertices can be separated. At each coarser pass, a complete block of the already updated finer partition becomes one atom:

$$\mathcal{G}_t^\ell = \{H\in\mathcal{P}_t^{\ell-1} : H\subseteq U_t^\ell\}. \tag{11}$$

Every pass starts from a fresh restriction of the stored adjacency,

$$\boldsymbol{A}_\ell^{\text{work}} = \boldsymbol{D}_\ell\boldsymbol{A}_t\boldsymbol{D}_\ell, \tag{12}$$

where $\boldsymbol{D}_\ell=\operatorname{diag}(\mathbf{1}_{U_t^\ell})$ masks rows and columns outside U_t^ℓ. This ambient $n\times n$ matrix is used with the membership matrix below; restricting the row and column indices to U_t^ℓ instead gives the locally indexed induced adjacency $\boldsymbol{A}_t[U_t^\ell]$. If $k_\ell=|\mathcal{G}_t^\ell|$, the membership matrix $\boldsymbol{P}_\ell\in\{0,1\}^{k_\ell\times n}$ contains one unit entry in each column indexed by U_t^ℓ and zero columns outside the update region. The backend adjacency is

$$\bar{\boldsymbol{A}}_\ell = \boldsymbol{P}_\ell\boldsymbol{A}_\ell^{\text{work}}\boldsymbol{P}_\ell^{\mathsf{T}}. \tag{13}$$

The backend therefore receives one vertex per atom and the total weight between every ordered pair of atoms.

For a feature-aware backend, ComNetX supports sum and mean aggregation. For $G_{\ell a}\in\mathcal{G}_t^\ell$,

$$\bar{\boldsymbol{x}}_{\ell a} = \begin{cases}\displaystyle\sum_{i\in G_{\ell a}}\boldsymbol{x}_i, & \text{sum},\\[2ex] \dfrac{1}{|G_{\ell a}|}\displaystyle\sum_{i\in G_{\ell a}}\boldsymbol{x}_i, & \text{mean}.\end{cases} \tag{14}$$

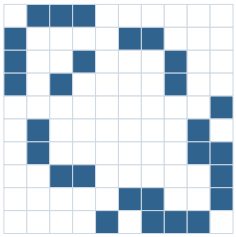
2) Lifting the Partition

The current implementation calls the backend without an initial quotient partition: the stored hierarchy determines the atoms but is not passed as $\boldsymbol{y}_0$. After the backend returns a quotient label vector, its induced blocks are lifted through $\boldsymbol{P}_\ell$. Each lifted block $H\subseteq U_t^\ell$ receives the canonical label $\kappa(H)=\min H$; labels outside U_t^ℓ remain unchanged. Algorithm 1 summarizes one batch update.

Stored state

1 Sparse adjacency

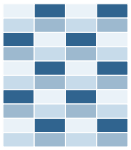
$A_t \in \mathbb{R}^{n \times n}$



Accumulated edge weights

2 Optional features

$X \in \mathbb{R}^{n \times d}$



Fixed vertex attributes

3 Partition hierarchy

$C_{t-1} \in \mathbb{Z}^{L \times n}$

1	2	1	1	5	2	2	1	2	2
1	1	1	1	5	1	1	1	1	1
1	1	1	1	5	1	1	1	1	1

Rows: fine to coarse
Columns: vertices 1 to 10

Detector inputs

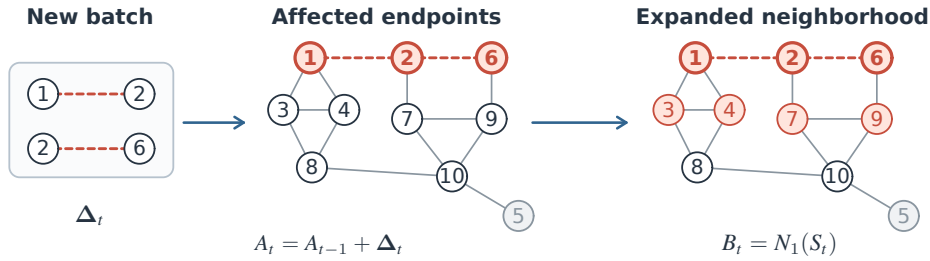
Topology only

$\bar{A}_\ell$

With features

$(\bar{A}_\ell, \bar{X}_\ell)$

1 Apply the batch and select update regions

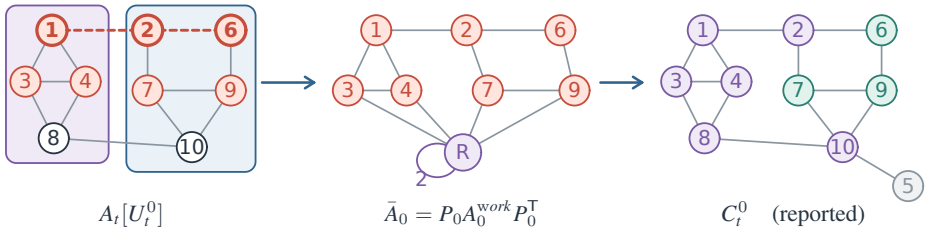


$S_t = \{1, 2, 6\}, \quad B_t = \{1, 2, 3, 4, 6, 7, 9\}$

Select all intersected communities at each level; fix all regions before the passes.

2 Contract, partition, and lift at level 0

$U_t^0 = \{1, 2, 3, 4, 6, 7, 8, 9, 10\}$



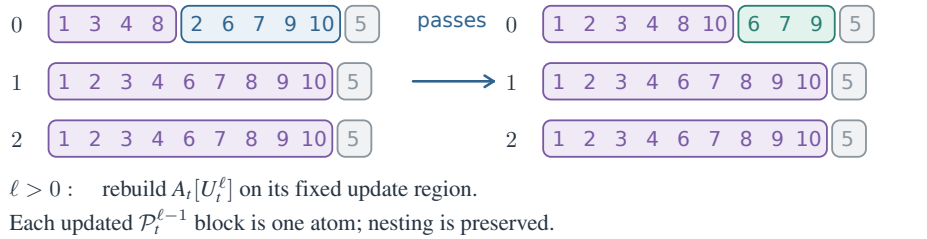
$R = \{8, 10\}; \quad |\mathcal{G}_t^0| = 8.$ Every vertex in B_t remains separate.

Aggregate features over atoms by sum or mean before the detector call.

Lift labels to original vertices; canonicalize blocks; keep vertex 5 unchanged.

---- new edge ○ vertex in B_t ○ outside U_t^0

3 Update the hierarchy from fine to coarse



$\ell > 0$: rebuild $A_t[U_t^\ell]$ on its fixed update region.

Each updated $\mathcal{P}_t^{\ell-1}$ block is one atom; nesting is preserved.

FIGURE 1. **ComNetX update procedure with $r = 1$. The left panel shows the stored adjacency, optional features, and partition hierarchy. (1) A batch identifies affected endpoints and their expanded neighborhood; all update regions are fixed from the pre-update hierarchy. (2) The first pass forms an induced graph, contracts atoms, calls the detector, and lifts its partition. Colored outlines mark two pre-update fine communities sharing a coarsest parent. Every vertex in B_t remains a singleton atom; only residual vertices 8 and 10 form R, whose internal edge gives quotient diagonal weight 2. (3) Each coarser pass uses complete updated finer blocks as atoms. The finest partition is reported. Shown backend outputs are illustrative; vertex 5 lies outside the update regions and retains its assignment.**

Algorithm 1 ComNetX update for one batch

Require: A_{t-1}, C_{t-1}, optional X, update Δ_t, radius r, backend $\mathcal{B}$, settings θ

$A_t \leftarrow A_{t-1} + \Delta_t$
$S_t \leftarrow$ endpoints of the updated edges; $B_t \leftarrow N_r(S_t)$
for $\ell = 0, \ldots, L-1$ **do**
  $U_t^\ell \leftarrow$ closure of B_t under $\mathcal{P}_{t-1}^\ell$ ▷ fixed before all passes
end for
$C_t \leftarrow C_{t-1}$
for $\ell = 0, \ldots, L-1$ **do** ▷ fine to coarse
  if $\ell = 0$ **then**
    $\mathcal{G}_t^\ell \leftarrow$ singleton atoms for B_t and the nonempty residual groups in (10)
  else
    $\mathcal{G}_t^\ell \leftarrow$ complete updated $\mathcal{P}_t^{\ell-1}$-blocks in U_t^ℓ
  end if
  $A_\ell^{\text{work}} \leftarrow D_\ell A_t D_\ell$
  build P_ℓ from $\mathcal{G}_t^\ell$
  $\bar{A}_\ell \leftarrow P_\ell A_\ell^{\text{work}} P_\ell^\top$
  aggregate $\bar{X}_\ell$ if $\mathcal{B}$ uses features
  $\bar{y}_\ell \leftarrow \mathcal{B}(\bar{A}_\ell, \bar{X}_\ell, \varnothing; \theta)$
  lift $\bar{y}_\ell$ to U_t^ℓ; set $C_t^\ell(v) \leftarrow \min H$ for every lifted block H and $v \in H$
end for
return C_t with reported partition C_t^0

D. STRUCTURAL PROPERTIES

1) Hierarchy Consistency

Starting from a canonical, nested hierarchy, Algorithm 1 preserves nesting:

$$\mathcal{P}_t^0 \preceq \mathcal{P}_t^1 \preceq \cdots \preceq \mathcal{P}_t^{L-1}. \tag{15}$$

Equation (9) places each update region at a finer level inside the corresponding coarser update region. Moreover, U_t^ℓ is a union of pre-update blocks at level ℓ and therefore, by pre-update nesting, a union of complete blocks at level $\ell - 1$. The finer pass changes only $U_t^{\ell-1}$; its lifted blocks stay inside that update region, while canonical labels prevent them from colliding with retained outside blocks. Since $U_t^{\ell-1} \subseteq U_t^\ell$, the coarser update region remains a union of complete updated blocks at level $\ell - 1$. Equation (11) makes each of these blocks one quotient vertex, so the coarser backend may merge but cannot split it. Ascending induction over ℓ gives (15). Adjacent hierarchy levels may coincide.

Canonical labels also make lifting invariant to backend label permutations: disjoint lifted blocks receive distinct original-vertex identifiers. Retaining labels outside U_t^ℓ yields a complete label vector after each pass.

2) Modularity under Contraction

The construction uses the standard modularity-preserving graph contraction [73]. To apply it to one update region, let $A_U \in \mathbb{R}^{|U|\times|U|}$ be the locally indexed adjacency induced by one update region, assume its total weight W_U is positive, and let $\mathcal{G} = \{G_1, \ldots, G_k\}$ be a partition of U. Define the local membership matrix $P_U \in \{0,1\}^{k\times|U|}$ and let $\bar{A} = P_U A_U P_U^\top$, retaining the self-loops created by aggregation. Then

$$\bar{A}_{ab} = \sum_{i\in G_a} \sum_{j\in G_b} A_{ij}. \tag{16}$$

Consequently, the quotient preserves total weight and the sums of outgoing and incoming weighted degrees. Every quotient partition Z induces a unique partition C of U that keeps each atom intact, and, under the same directed or symmetric adjacency, self-loop, and resolution conventions,

$$Q_\gamma(\bar{A}, Z) = Q_\gamma(A_U, C). \tag{17}$$

The identity follows by expanding each quotient entry in the internal-weight and degree terms of (5). It holds for partitions that keep each atom intact and does not imply preservation of an arbitrary feature-aware objective.

Edges crossing from U_t^ℓ to the rest of the graph are omitted from the backend input. Its modularity ranking can therefore differ from the full-graph ranking. The exact decomposition in Appendix A attributes this discrepancy to boundary degrees and global-versus-local null-model normalization. We therefore evaluate every returned partition on the full graph.

E. TIME AND MEMORY COMPLEXITY

For $r = 1$, let $d_t(v)$ denote the degree of v in the symmetrized support graph. Then

$$|B_t| \le |S_t| + \sum_{v\in S_t} d_t(v). \tag{18}$$

Thus, a small edge batch can still have a large radius-one expansion when it touches hubs. If $\mathcal{P}_{t-1}^\ell$ is the pre-update partition at level ℓ, the update-region size $h_\ell = |U_t^\ell|$ is exactly

$$h_\ell = \sum_{H\in\mathcal{P}_{t-1}^\ell : H\cap B_t \neq \varnothing} |H|. \tag{19}$$

The batch size, endpoint degrees, and sizes of the intersected communities determine different parts of the workload. Let $r_t \le r$ be the number of neighborhood traversals, s_t the number of stored adjacency entries, e_ℓ the number of entries in $A_t[U_t^\ell]$, and k_ℓ and q_ℓ the numbers of quotient vertices and stored entries in the quotient adjacency matrix. Let $\overline{T}_{\mathcal{B}}(k,q,d)$ and $\overline{M}_{\mathcal{B}}(k,q,d)$ be nondecreasing worst-case envelopes for the core backend time and working memory, excluding format conversion. Assume that conversion of a quotient to the backend representation is linear in its stored adjacency and feature arrays; these terms are dominated below by the quotient-construction and feature terms. The following bound assumes direct endpoint relabeling and comparison-based sort-and-reduce; the current PyTorch realization is analyzed separately below and in Appendix B.

1) Worst-Case Resources

For a feature-aware call, let $F = nd$, a valid upper bound for dense or sparse feature storage; for a topology-only call, let $F = 0$. Suppose quotient edges are assembled by relabeling their endpoints followed by comparison-based sort-and-reduce and levels are processed sequentially. After the batch has been applied, one update can be implemented in

$$T_{\text{update}} = O\Big(r_t(n+s_t) + L\big[(n+s_t)\log(2+n+s_t) + F\big] + \sum_{\ell=0}^{L-1} \overline{T}_{\mathcal{B}}(k_\ell, q_\ell, d)\Big) \tag{20}$$

time and

$$M_{\text{peak}} = O\Big(s_t + Ln + F + \max_{\ell}\{e_\ell + h_\ell + q_\ell + \overline{M}_{\mathcal{B}}(k_\ell, q_\ell, d)\}\Big) \tag{21}$$

memory. The maintained state alone occupies $O(s_t + Ln + F)$ memory. The time bound follows from at most r_t sparse traversals, L scans of the label and edge arrays, sort-and-reduce construction of at most s_t quotient entries per level, at most one feature aggregation and linear format conversion per level, and the L backend calls. The space bound follows because only one level, converted quotient, and backend call need to be materialized at a time.

Since $h_\ell, k_\ell \le n$ and $e_\ell, q_\ell \le s_t$, a nondecreasing backend cost gives the explicit graph-wide worst case

$$T_{\text{worst}} = O\Big(r_t(n+s_t) + L\big[(n+s_t)\log(2+n+s_t) + F + \overline{T}_{\mathcal{B}}(n, s_t, d)\big]\Big). \tag{22}$$

Thus ComNetX has no unconditional asymptotic speedup: a hub-centered update or a large intersected community can make every local problem comparable to the full snapshot, and up to L backend calls are then required.

2) Costs of the Present Implementation

The present PyTorch implementation uses tensor sparse primitives rather than the sort-and-reduce realization assumed in (20). At every hierarchy level, it scans the full label arrays and filters the full stored edge array. Conditional on executing all r_t traversals and L passes, it therefore incurs

$$\Omega(r_t s_t + L(n+s_t)) \tag{23}$$

full-scan memory traffic even when the quotient graphs are small. These global scans are a property of the current implementation rather than a requirement of the update rule. Appendix B gives its temporary-memory requirements.

Contraction is beneficial when the detector work saved exceeds the cost of neighborhood search, closure, restriction, aggregation, lifting, and format conversion. This balance depends on the backend and on $|B_t|$, h_ℓ, e_ℓ, k_ℓ, and q_ℓ. The experiments examine these sizes alongside processing time and partition quality.

V. EXPERIMENTAL DESIGN

We organize the evaluation around five questions.

RQ1 What quality–time trade-off does ComNetX achieve relative to warm-started full-snapshot recomputation?

RQ2 How does this trade-off change with the wrapped backend, including feature-aware calls under backend-specific interface rules and a specialized dynamic implementation?

RQ3 How do radius expansion and community closure change the update region, and how does quotient contraction determine the final backend input?

RQ4 How sensitive is the measured quality–time trade-off to hierarchy depth, radius, feature representation, aggregation, and random-feature initialization?

RQ5 How does the trade-off evolve over longer update sequences, and how does it depend on update granularity and the fraction of the graph already available at initialization?

A. DATASETS AND STREAM CONSTRUCTION

Table 1 lists the six input graphs. Cora, ACM, CiteSeer, and PubMed are static attributed topologies converted into reproducible update benchmarks with artificial edge orders. Patent and ArxivMath use timestamped sequences from temporal graph-clustering resources [74], [75]; their records are sorted by the provided timestamps. We distinguish source-graph objects from stream records: an undirected edge is an unordered vertex pair, a directed arc is an ordered pair, and a stored record is one entry in the input sequence. The source counts in Table 1 count each undirected edge once for Cora, ACM, and CiteSeer, and count directed arcs for PubMed, Patent, and ArxivMath. All protocol fractions and batch sizes below refer to stored records, not to distinct undirected edges in the symmetrized graph.

For reproducibility, Cora, ACM, and CiteSeer can be obtained from the open dataset collection associated with [65]. The Cora and CiteSeer topologies match the preprocessed Planetoid graphs [76]. PubMed uses the 44,338-arc variant distributed through PyTorch Geometric's `AttributedGraphDataset`.[1] The artifact documents exact input comparisons and the stored edge order.

Stream construction partitions stored edge records. Cora, ACM, and CiteSeer store both orientations of each undirected edge, and the loader symmetrizes each update. When a tail is subdivided, the two records of an edge can occur in different batches: the first inserts a symmetric unit-weight edge and the second raises its weight to two. For example, Cora's ten short-protocol updates process five distinct undirected edges. Full and ComNetX runs receive identical weighted snapshots. Thus the artificial streams test additive edge updates, including weight increments, rather than only previously absent edges.

[1] https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.datasets.AttributedGraphDataset.html

TABLE 1. Source graphs and actual updates under 999:10. Source counts are undirected edges for Cora, ACM, and CiteSeer, and directed arcs for the other graphs. The protocol operates on stored records.

Dataset	Vertices	Source count	Order	Updates
Cora	2,708	5,278	artificial	10
ACM	3,025	13,128	artificial	10
CiteSeer	3,327	4,552	artificial	8
Patent	12,214	41,916	timestamp	10
PubMed	19,717	44,338	artificial	10
ArxivMath	270,013	799,745	timestamp	10

A real-graph protocol p:q partitions the stored record sequence into $p+1$ coarse bins of approximately equal record counts, uses the first p bins as the initial graph, and splits the tail bin into at most q chunks whose record counts differ by at most one. Empty chunks are skipped. The primary 999:10 protocol therefore measures ten updates on five graphs and eight on CiteSeer. It tests maintenance of an almost complete graph. The short Leiden comparison contains Cora, ACM, CiteSeer, PubMed, and ArxivMath; the archive contains no Patent Leiden execution for this protocol. S^{2}CAG covers all six graphs. The growth comparison instead contains Cora, ACM, CiteSeer, Patent, and PubMed: an ArxivMath full run is present, but its ComNetX record is empty. These subsets reflect the availability of complete pairs; the archive does not record the reasons for the missing executions.

We distinguish three complementary controls. First, 999:q with $q \in \{5, 10, 20, 40, 80, 160, 320\}$ varies granularity while retaining the same initial graph and tail records. The PubMed tail contains 44 records and the ArxivMath tail 799, so the number of nonempty updates is bounded by these counts. Second, 99:50 and 9:500 reserve approximately 1% and 10% of the records for updates. The latter supplies 500 nonempty updates on both graphs and tests sustained maintenance over a larger evolving portion of the graph. Third, protocol 0:10 divides the entire record sequence into ten bins and processes all ten, including the first snapshot, without a separate Leiden bootstrap. It tests graph growth from singleton initialization rather than maintenance of an almost complete graph. The archived launcher records this protocol using the legacy key `10`; the notation 0:10 is used throughout the article.

The original campaign measures each method and parameter/feature configuration once. A separate S^{2}CAG campaign repeats each full/local pair five times under fixed inputs and seeds. Five feature seeds examine random-input sensitivity on the same graph streams, rather than repeated executions with identical inputs. Protocols that subdivide the same record sequence are controlled comparisons, not independent temporal windows. Table 2 summarizes the design.

B. TIMING AND COMPUTING ENVIRONMENT

The principal efficiency measure is cumulative processing time. The full and ComNetX paths use wall-clock elapsed time minus the adapter-reported conversion and preprocessing interval. Native DF-Leiden instead supplies the duration returned by its `apply()` call, after its exposed `update()` call. Table 3 specifies the boundaries in the preserved benchmarking code. These intervals are not end-to-end batch latency. The native and ComNetX intervals are instrumented differently, so their ratios compare the reported execution paths. Speedup is the full or native-dynamic time divided by ComNetX time for the paired dataset and protocol.

For S^{2}CAG, feature preparation and graph normalization are subtracted, but dense feature conversion, all five solver calls, and internal accuracy, NMI, and adjusted Rand index (ARI) calculations remain timed. Only the external metrics computed by the benchmarking script are excluded. The repeated S^{2}CAG campaign also records the interval before this subtraction, as described in Section V-C4.

The original timing campaign ran on a Dockerized Linux server with two AMD EPYC 7742 64-core central processing units (CPUs), 2.0 TiB of random-access memory (RAM), and eight NVIDIA A100-SXM4 graphics processing units (GPUs; 80 GB each). The NVIDIA driver was 535.216.03, `nvidia-smi` reported CUDA 12.2, and the container used Python 3.10.13, PyTorch 2.3.1 with CUDA 11.8, and TensorFlow 2.14.0.

The repeated S^{2}CAG campaign used two EPYC 7742 CPUs, one visible A100-SXM4 GPU (80 GB), Python 3.10.13, PyTorch 2.3.1 with CUDA 12.1, and TensorFlow 2.16.2. Graph tensors used CUDA; the spectral solver ran on the CPU with eight threads. The artifact records the complete environment and hashes of the executed sources separately for this campaign.

For RQ3, separate instrumented CPU executions record the neighborhood, update-region, and contracted-graph sizes on Cora, PubMed, and ArxivMath. They use 999:10, $L = 3$, $r = 1$, the public `leidenalg` backend, and the same stream construction. Their initial hierarchies and software versions are included in the accompanying artifact. Only graph dimensions from these executions are reported; their timings are not combined with the server comparisons.

The topology-parameter and feature controls use separate experiments. Each control is interpreted within its stated protocol and is not pooled with the main comparison.

C. METHODS AND FIXED CONFIGURATION

Leiden is the primary backend and uses resolution 1. Full recomputation receives the previous partition as a warm start. ComNetX uses $L = 3$, $r = 1$, and the Leiden initialization described in Section IV for p:q protocols with $p > 0$. The 0:10 growth control instead starts every hierarchy row from singletons and includes processing the first snapshot in its measured time. ComNetX does not pass an initial partition to quotient calls: the stored hierarchy determines the contracted groups only. The DF-Leiden baseline retains one native streaming object across batches, whereas ComNetX creates a fresh instance for each contracted graph. The original DF-Leiden

TABLE 2. Evaluation protocols: 999:q (including 999:10), 99:50, 9:500, and 0:10. Original settings have one execution per method; the repeated S^{2}CAG series has five. The last column distinguishes execution counts from varied controls.

Purpose	Graphs	Protocol	Actual updates	Runs and controls
Main Leiden comparison	five real topologies	999:10	8–10	one paired execution
Feature-aware backend	six real topologies	999:10	8–10	dataset/random features
Repeated S^{2}CAG pairs	PubMed, ArxivMath	999:5, 999:10	5, 10	five fixed-input repeats
Topology parameters	Cora, PubMed, ArxivMath	999:10	10	12 settings per graph
Graph-size diagnostics	Cora, PubMed, ArxivMath	999:10	10	separate CPU executions
S^{2}CAG feature controls	Cora, PubMed	999:10	10	modes, aggregation, radius
Feature-input sensitivity	six real topologies	999:10	8–10	five feature seeds
Batch granularity	PubMed, ArxivMath	999:q	5–320	seven requested chunk counts
Larger update windows	PubMed, ArxivMath	99:50, 9:500	50, 500	one edge order per graph
Graph growth	five paired real topologies	0:10	10	singleton initialization

TABLE 3. Measurement boundaries in the preserved benchmarking code. "Excluded" means outside the timer or explicitly subtracted. In the graph-growth experiments there is no separate bootstrap; processing the first snapshot is included.

Stage	Full	ComNetX	Native DF
Data loading and initial feature generation	Excluded	Excluded	Excluded
Initial partition/bootstrap	Excluded	Excluded	Excluded
Applying the batch / exposed update call	Excluded	Excluded	Excluded
Neighborhood expansion and community closure	–	Included	Internal[a]
Induced subgraph, quotient, aggregation, lifting	–	Included	Internal[a]
Adapter conversion/preprocessing interval	Subtracted	Subtracted	–
Detector optimization	Included	All passes	Reported time[a]
S^{2}CAG densification and five solver calls	Included	Included	–
S^{2}CAG internal accuracy/NMI/ARI	Included	Included	–
External modularity and reference-label metrics	Excluded	Excluded	Excluded

[a]Native DF-Leiden reports the duration of `apply()`; the binding does not expose its internal stages. The table does not assume that its internal work matches the ComNetX stages.

implementation is publicly available.[2]

All reported pairs are matched by dataset, update protocol, and graph representation. LD-Leiden is discussed as the closest Leiden-specific alternative but is not included in the quantitative benchmark. Its available implementation cannot be distributed with the artifact, and its native initialization and timing interfaces do not match the paired protocol used here; we therefore avoid a cross-protocol performance ranking.

1) S^{2}CAG Solver Settings

The preserved adapter uses `sub`, with propagation order $T = 10$, $\alpha = 1$, $\tau = 7$ singular value decomposition (SVD) iterations, ten oversampling columns, and 100 rounding iterations. The argument `gamma=1` is unused by `sub`; it does not select the separate `mod` branch. Modularity is evaluated externally at resolution one.

Each invocation makes five solver calls and returns the last result. Both SVD implementations default to seed zero, so these are not independent random restarts. The printed internal standard deviations use one observation collected after the loop and are not used to assess stability. The feature-seed study below instead varies five input-feature initializations.

[2]https://github.com/puzzlef/leiden-communities-openmp-dynamic

2) Requested and Realized Cluster Counts

In a full S^{2}CAG call, the previous labels determine only the requested cluster count k; they do not initialize the spectral solver or its rounding procedure. Thus this call is not warm-started. A local call receives no prior labels and requests as many clusters as there are quotient vertices. For an input with n vertices and d feature columns, the implemented singular-vector slice `U[:, 1:k+1]` discards the first vector and retains s columns, where $s \le \min\{k, \min(n, d) - 1\}$. Rounding assigns rows to these columns and may leave some columns empty. The number of nonempty communities can therefore be smaller than both s and the requested k. This bound applies to each backend call. ComNetX also retains communities outside the update region, so its reported full-graph partition can contain more groups. The original records do not store realized counts or output label vectors; the repeated paired experiments below record both. These details distinguish the implemented adapter from an experiment with a fixed, equal number of communities in the full and local paths.

3) Feature Inputs and Preprocessing

The main comparisons use paired undirected representations. In the feature ablation, Cora uses its stored undirected representation and PubMed is symmetrized; each full/local pair uses the same representation. Local feature-aware calls

use mean aggregation unless the sum-aggregation control is specified. The retained presets for the main comparison and the feature, aggregation, and radius controls specify 64-dimensional random features; their omitted seed resolves to the loader default 42. The seed-study presets explicitly use 64 dimensions and seeds 1–5. The loader generates an $n \times 64$ float32 matrix with independent $N(0, 1)$ entries using a CPU `torch.Generator`, once per loaded graph. These presets document the original campaign settings; the repeated campaign records its actual feature configuration and input hashes separately.

Local S^{2}CAG calls first aggregate signed features over atoms, then apply elementwise absolute value, followed by term frequency–inverse document frequency (TF–IDF) weighting and row L2 normalization. Positive and negative values can therefore cancel inside an atom before the absolute value is taken. Full calls apply the absolute value directly to the original features. The one-hot control supplies an identity feature matrix, whose rows are aggregated using the same atom membership.

4) Repeated Paired Experiments

A separate campaign evaluates PubMed and ArxivMath under 999:5 and 999:10, using five full/ComNetX pairs per setting. It uses the same fixed S^{2}CAG solver settings, 64-dimensional random features with seed 42, and ComNetX with $L = 3$, $r = 1$, and mean aggregation. A shared initial hierarchy is constructed with public `leidenalg`, resolution one, seed one, and two iterations. The same initial hierarchy and record sequence are reused across the two subdivisions and all repeats. The initial finest partitions contain 55 communities on PubMed and 8,167 on ArxivMath.

Each method runs in a separate process; the order of full and local execution alternates between pairs. A small solver warm-up, initialization, batch application, external metrics, and label-file writes are excluded from the processing interval. Both intervals synchronize the tensor device and include instrumentation overhead. The main interval subtracts adapter preprocessing, as in Table 3; the second retains it. S^{2}CAG densification, all five internal calls, and internal metrics are included in both. Times are reported as mean $\pm$ sample standard deviation over the five executions, and speedup as the median of their paired full/local ratios. These fixed-input repetitions measure runtime repeatability, not variation across graph realizations or random-feature seeds.

VI. RESULTS AND DISCUSSION

A. QUALITY AND PROCESSING TIME ON SHORT MAINTENANCE STREAMS (RQ1)

ComNetX substantially reduces the measured cost of repeated Leiden optimization across the five short-stream comparisons in Table 4. The speedup ranges from 6.60$\times$ to 98.72$\times$, while the absolute final-modularity difference remains between 0.0035 and 0.0213. The largest graph, ArxivMath, combines a reduction from 110.4 s to 1.118 s with final modularity of 0.8812 compared with 0.8858 for full recomputation.

The quality differences are graph dependent. ACM has the largest modularity difference, whereas CiteSeer has the smallest. NMI decreases in all five pairs, with an almost unchanged value on CiteSeer and a larger difference on ArxivMath. The time gains should therefore be interpreted together with both quality measures: closeness in modularity does not imply identical partitions or identical agreement with reference labels.

B. RESULTS WITH DIFFERENT COMMUNITY-DETECTION BACKENDS (RQ2)

The native DF-Leiden comparison is reported with the sustained-maintenance experiments in Table 12. To evaluate the feature-aware interface, Table 5 reports S^{2}CAG on all six graphs with dataset and random features. With dataset features, ComNetX reduces measured processing time on five graphs, including a 51.72-fold reduction on ArxivMath from 4151.6 s to 80.270 s. Final NMI increases on five graphs as well.

The full and local S^{2}CAG calls use the cluster-count rules specified in Section V. Their time ratios consequently describe the implemented interface rather than equal-cluster-count optimization. With random features, full-run modularity is near zero, whereas ComNetX retains substantially higher values. The local procedure combines the feature input with graph structure and the stored hierarchy; the comparison does not attribute the quality difference to features alone. Patent with dataset features is the exception to the time reduction: ComNetX takes 13.474 s versus 11.390 s, while increasing both modularity and NMI.

1) Repeated S^{2}CAG Comparisons

Table 6 reports the separate repeated campaign. Every pair is faster with ComNetX under both timing definitions. With preprocessing subtracted, median speedups are 3.44 and 3.37 on PubMed and 37.63 and 70.40 on ArxivMath for 999:5 and 999:10, respectively. Including preprocessing gives 3.52, 3.46, 37.64, and 70.34 in the same order. Thus the time advantage in this campaign is not an artifact of subtracting preprocessing. These ratios are computed within the new pairs; the original campaign times in Table 5 are not pooled with them.

On ArxivMath, the first full-graph update requests $k = 8167$ and returns 16 and 15 groups under 999:5 and 999:10, respectively. It accounts for 93.64% and 90.85% of cumulative full processing time. The implemented randomized SVD uses $k + 10$ projection columns before truncating the retained embedding to at most 63 columns. The aggregate speedups therefore depend strongly on the adapter's initial cluster-count request. As a post hoc timing diagnostic, summing only updates $2, \ldots, q$ for both methods gives median paired speedups of 3.06 and 7.46, respectively (3.12 and 7.65 with preprocessing included).

TABLE 4. Final quality and cumulative measured processing time for full-snapshot and ComNetX Leiden on 999:10 streams.

Dataset	Full Q	Local Q	Full NMI	Local NMI	Full T (s)	Local T (s)	Speedup
Cora	0.8240	0.8141	0.4667	0.4507	0.982	0.149	6.60
ACM	0.8255	0.8042	0.2700	0.2659	0.930	0.123	7.54
CiteSeer	0.8932	0.8897	0.3305	0.3304	0.647	0.077	8.43
PubMed	0.7872	0.7822	0.2067	0.1983	4.727	0.157	30.12
ArxivMath	0.8858	0.8812	0.4226	0.4070	110.4	1.118	98.72

TABLE 5. Main S^{2}CAG feature comparison under 999:10; quality and time values are full-snapshot/ComNetX. Local calls use $L = 3, r = 1$ and mean aggregation. Each setting has one paired execution. Table 6 reports the separate repeated campaign.

Dataset	Features	Q	NMI	T (s)	Time ratio
Cora	dataset	0.6951/0.7859	0.4355/0.4504	29.949/5.553	5.39
ACM	dataset	0.0536/0.6219	0.0814/0.2538	224.2/10.713	20.93
CiteSeer	dataset	0.7770/0.7719	0.2542/0.3262	135.8/1.792	75.83
Patent	dataset	0.3970/0.5520	0.0604/0.3560	11.390/13.474	0.85
PubMed	dataset	0.6221/0.4987	0.1686/0.1524	78.869/13.609	5.80
ArxivMath	dataset	0.0003/0.0844	0.0002/0.1410	4151.6/80.270	51.72
Cora	random	-0.0020/0.7877	0.0233/0.4505	9.408/4.357	2.16
ACM	random	0.0004/0.6222	0.0089/0.2567	11.286/5.256	2.15
CiteSeer	random	-0.0031/0.7719	0.0170/0.3262	9.519/0.073	129.81
Patent	random	-0.0055/0.6579	0.0009/0.3560	10.165/8.802	1.15
PubMed	random	0.0027/0.5010	0.0003/0.1580	18.272/6.199	2.95
ArxivMath	random	0.0012/0.5875	0.0002/0.1769	3889.9/63.597	61.16

TABLE 6. Repeated paired S^{2}CAG experiments under 999:5 and 999:10, with random features, $L = 3$, $r = 1$, and mean aggregation. Q, NMI, and K are full/ComNetX final values; K counts nonempty communities in the reported partition of the entire graph. Each method returns identical Q, NMI, and K in its five repeats at a given setting. Times exclude adapter preprocessing and are mean $\pm$ sample standard deviation in seconds. S is the median of the five paired time ratios, not the ratio of the displayed mean times.

Dataset	Protocol	Q	NMI	K	Full T (s)	ComNetX T (s)	S
PubMed	999:5	0.0001/0.4693	0.0003/0.0869	11/206	11.995±0.101	3.526±0.063	3.44
PubMed	999:10	0.0001/0.4683	0.0003/0.1112	11/392	20.342±0.095	6.020±0.065	3.37
ArxivMath	999:5	0.0020/0.5724	0.0003/0.1909	13/8,119	3819.065±21.885	101.379±0.563	37.63
ArxivMath	999:10	0.0024/0.5633	0.0003/0.1774	13/8,116	3951.633±37.385	56.371±0.196	70.40

Final Q, NMI, and group counts agree across fixed-input repeats. ComNetX improves both quality scores relative to full S^{2}CAG, but the numbers of output communities differ substantially: the final full-graph partitions contain 11 or 13 groups, whereas ComNetX reports 206–392 on PubMed and more than 8,100 on ArxivMath. This is a comparison under the adapter's different cluster-count rules for full and local calls; it does not establish an advantage at equal numbers of communities.

For PubMed, increasing q from 5 to 10 raises mean cumulative ComNetX time from 3.526 to 6.020 s. For ArxivMath, ComNetX time instead falls from 101.379 to 56.371 s, while final Q decreases from 0.5724 to 0.5633 and NMI from 0.1909 to 0.1774. The ArxivMath time reduction is concentrated in solver calls at level zero: their mean cumulative time falls from 97.472 to 51.222 s, with quotient sizes of 4,327–4,976 and 2,128–3,049 vertices, respectively. The traces locate the reduction in solver processing of smaller quotients; they do not separate costs within the spectral solver.

C. SIZE OF THE CONTRACTED GRAPHS (RQ3)

Table 7 reports neighborhood, update-region, and quotient sizes from the separate CPU executions. On PubMed and ArxivMath, the update region at level 0 contains, on average, 64.3% and 78.7% of all vertices, although the radius-one neighborhoods contain only 154.9 and 782.0 vertices. Community closure can therefore select a substantial part of a graph even for small batches. Contraction reduces these regions to an average of 167.9 and 825.3 vertices, respectively; the later passes use still smaller graphs. Thus a small contracted graph does not require a small update region.

The equality of k_1 and k_2 is present in every one of the 30 update records, before averaging. The corresponding update-region and quotient-edge counts also agree. These executions have coincident initial hierarchy levels, and the second pass does not further reduce the number of atoms supplied to the

third pass. Equal adjacent levels are allowed; $k_1 = k_2$ is not a general property of ComNetX.

The cost analysis expresses detector work in terms of k_ℓ and q_ℓ, separately from the cost of selecting and lifting the update region. The implementation still scans full label and edge arrays, as described in Section IV. These size measurements characterize the constructed inputs; they do not isolate the timing contribution of each stage. The depth and radius study below examines the resulting quality–time trade-off.

D. SENSITIVITY TO HIERARCHY DEPTH, RADIUS, AND FEATURES (RQ4)

Figure 2 shows all 36 tested depth–radius configurations. Every local point is faster than its full-snapshot baseline. The ranges are 8.52–23.42× on Cora, 7.63–125.54× on PubMed, and 15.27–636.84× on ArxivMath. The nondominated configurations differ across graphs in both hierarchy depth and radius.

On PubMed, $L = 2, r = 0$ yields modularity 0.7847 at 72.88× speedup, compared with full modularity of 0.7872. The highest local modularity on this graph, 0.7851, occurs at $L = 2, r = 2$. On ArxivMath, the fastest configuration, $L = 1, r = 0$, reaches 636.84× with modularity 0.8792; the higher-quality $L = 2, r = 2$ configuration reaches modularity 0.8819. Cora attains its highest local modularity at $L = 3, r = 2$. Neither a larger radius nor a deeper hierarchy consistently improves modularity. The common $L = 3, r = 1$ setting is used for consistency across datasets; it is not selected as the best configuration for each graph.

1) Effect of Feature Representation

Table 8 reports a separate feature-ablation execution series with the same solver settings as Table 5. The overlapping dataset/random-feature settings on Cora and PubMed have identical final Q and NMI at stored precision, but independently recorded times. Each table uses the full and local runs from its own series; these timings are not pooled.

On Cora, ComNetX maintains similar modularity across all three feature modes, 0.7859–0.7877. Random features give near-zero modularity in the full calls, yet the corresponding ComNetX results remain close to the local dataset-feature results: 0.7877 versus 0.7859 on Cora and 0.5010 versus 0.4987 on PubMed. With the one-hot input mode, ComNetX reduces the PubMed processing time from 1973.0 s to 120.0 s. These results show that the computational benefit can persist across the tested input representations, while the effect on partition quality depends on the graph and the backend's feature and cluster-count rules.

2) Feature Aggregation and Radius

The matching quality scores in Table 9 are expected under the S^{2}CAG preprocessing used here. For each nonempty atom a, $\boldsymbol{x}_{\mathrm{mean}}(a) = \boldsymbol{x}_{\mathrm{sum}}(a)/|a|$. Taking absolute values preserves this positive scale factor and leaves the feature supports, hence the IDF weights, unchanged. Linear TF–IDF weighting preserves the factor, and row L2 normalization cancels it; zero rows remain zero. Thus mean and sum yield the same normalized features in exact arithmetic, although floating-point rounding can introduce small differences. The single-execution timings do not establish a systematic runtime difference.

In Table 10, radius zero yields the highest modularity on both graphs. Increasing the radius to two reduces PubMed modularity from 0.6601 to 0.1659. Radius changes both the selected vertices and their aggregated representation; its effect on quality must therefore be assessed alongside its computational cost.

3) Variation across Random-Feature Seeds

Table 11 summarizes five random-feature seeds under the common local configuration. Cora, ACM, and CiteSeer exhibit small variation in final quality. Patent has a modularity standard deviation of 0.0522, exceeding PubMed's 0.0434, while its NMI remains nearly constant (standard deviation 0.0003). PubMed and ArxivMath vary in both quality measures. Thus feature-input sensitivity depends on both the dataset and the metric. The reported standard deviations characterize these runs on fixed graph streams, not variation across independent graph realizations.

E. LONGER STREAMS, UPDATE GRANULARITY, AND INITIALIZATION (RQ5)

1) Sustained Maintenance over 500 Updates

ComNetX retains a substantial time advantage when a tenth of the stored record sequence is processed over 500 updates. Under 9:500, Leiden processing time falls from 248.6 s to 10.165 s on PubMed and from 5652.7 s to 77.788 s on ArxivMath, corresponding to 24.46-fold and 72.67-fold speedups. Table 12 reports final quality and time for both topology backends. For Leiden, the final modularity gaps are 0.0078 on PubMed and 0.0062 on ArxivMath. PubMed NMI increases from 0.2017 to 0.2045; ArxivMath NMI changes from 0.4205 to 0.4006. Thus PubMed's higher NMI accompanies a small modularity decrease, whereas both scores decrease on ArxivMath.

Fig. 3 shows that the absolute modularity difference remains below 0.008 at every measured update on both graphs. The mean local-minus-full difference is -0.00584 on PubMed and -0.00524 on ArxivMath; the minima are -0.00790 at update 477 and -0.00617 at update 500, respectively. The cumulative time ratio remains above one throughout both trajectories.

Against native DF-Leiden, the time advantage depends on the graph. ComNetX with DF-Leiden reduces ArxivMath time from 134.4 s to 13.517 s, while increasing final modularity from 0.8551 to 0.8792 and NMI from 0.3884 to 0.3967. On PubMed, a 0.354 s increase in processing time accompanies higher modularity (0.7786 versus 0.7619) and NMI (0.2051 versus 0.1946).

TABLE 7. Measured graph dimensions, averaged over ten updates in the separate CPU diagnostics (999:10, $L = 3$, $r = 1$). Parentheses give percentages of all n vertices. Here $h_0 = |U_t^0|$, k_ℓ is the number of contracted vertices at level ℓ, and q_0 counts stored adjacency entries, including self-loops and both orientations of symmetric edges. The final column combines k_1 and k_2, which coincide in all 30 update records of this series.

Dataset	$\lvert B_t \rvert$	h_0	k_0	q_0	$k_1 = k_2$
Cora	9.4	314.2 (11.6%)	11.6 (0.43%)	45.6	3.0
PubMed	154.9	12,686.8 (64.3%)	167.9 (0.85%)	1,393.9	13.0
ArxivMath	782.0	212,558.8 (78.7%)	825.3 (0.31%)	7,408.8	47.4

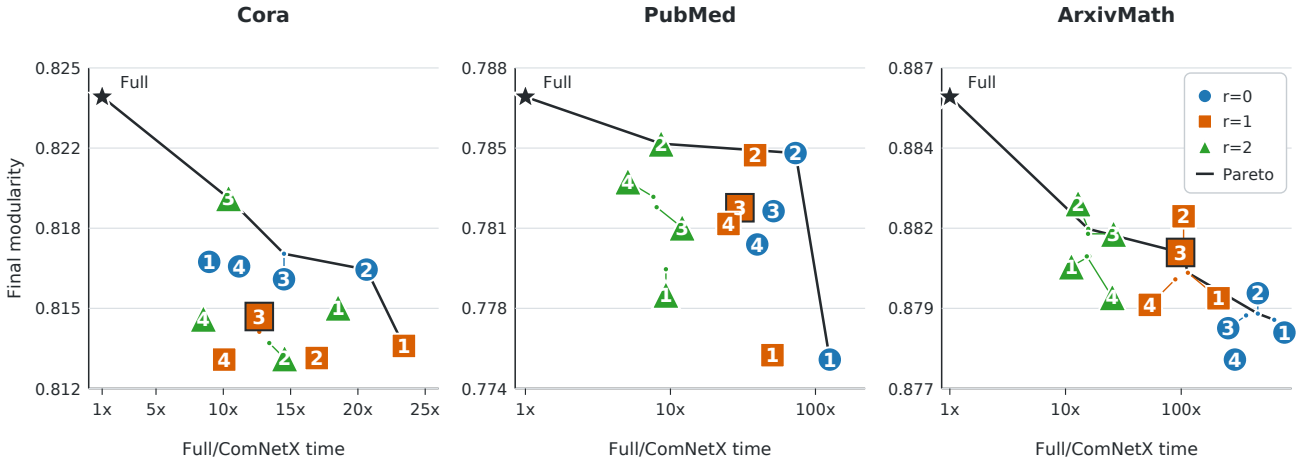


FIGURE 2. Effects of hierarchy depth and radius on Leiden modularity and processing time under 999:10, with $L \in \{1, 2, 3, 4\}$ and radius $r \in \{0, 1, 2\}$. Marker numbers show L; color and shape show r; and the black line connects nondominated measured points. The outlined orange square is the fixed $L = 3, r = 1$ configuration. Each point is one execution; ratios use the full baseline from the same parameter study.

TABLE 8. S^{2}CAG feature sensitivity under 999:10, with ten updates on each graph. Each cell reports Q / NMI / T (seconds). These executions are separate from the main feature comparison in Table 5.

Variant	Cora	PubMed
Full, dataset features	0.6951/0.4355/30.156	0.6221/0.1686/78.119
ComNetX, dataset features	0.7859/0.4504/5.407	0.4987/0.1524/14.394
Full, random features	-0.0020/0.0233/8.571	0.0027/0.0003/18.861
ComNetX, random features	0.7877/0.4505/4.240	0.5010/0.1580/6.017
Full, one-hot input	0.6974/0.3528/35.310	0.6540/0.1071/1973.0
ComNetX, one-hot input	0.7862/0.4500/5.632	0.5349/0.1884/120.0

TABLE 9. S^{2}CAG feature aggregation under 999:10, at $L = 3, r = 1$, using random features.

Dataset	Aggregation	Q	NMI	T (s)
Cora	mean	0.7877	0.4505	5.929
Cora	sum	0.7877	0.4505	4.197
PubMed	mean	0.5010	0.1580	6.177
PubMed	sum	0.5010	0.1580	5.995

TABLE 10. S^{2}CAG radius sensitivity under 999:10, at $L = 3$, with random features and mean aggregation.

Dataset	r	Q	NMI	T (s)
Cora	0	0.8156	0.4509	1.740
Cora	1	0.7877	0.4505	4.368
Cora	2	0.6541	0.4120	4.829
PubMed	0	0.6601	0.1350	4.386
PubMed	1	0.5010	0.1580	6.248
PubMed	2	0.1659	0.0329	30.304

2) Subdivision of a Fixed Update Window

Table 13 varies the number of chunks in the final 0.1% of each stored record sequence. Finer subdivision increases the number of full-graph recomputations while reducing the number of records per update. The $q = 10$ row repeats the nominal short-maintenance setting in a separate execution series and uses its own full baseline. Thus PubMed's time ratio of 31.42 here differs from 30.12 in Table 4, although final Q and NMI coincide. This difference does not represent a change in parameters. On ArxivMath, the Leiden speedup

TABLE 11. ComNetX with S^2CAG and random features under 999:10: mean ± sample standard deviation across feature seeds 1–5, with $L = 3, r = 1$ and mean aggregation.

Dataset	Q	NMI	T (s)
Cora	0.7853±0.0014	0.4499±0.0012	5.128±1.890
ACM	0.6211±0.0018	0.2580±0.0017	4.200±2.300
CiteSeer	0.7719±0.0000	0.3262±0.0000	1.415±0.753
Patent	0.6343±0.0522	0.3562±0.0003	6.962±3.834
PubMed	0.4746±0.0434	0.1399±0.0327	4.945±2.709
ArxivMath	0.5432±0.1034	0.2162±0.1061	50.524±28.162

TABLE 12. Quality and measured processing time after 500 updates under 9:500. Values are baseline/ComNetX; the baseline is full Leiden or native dynamic DF-Leiden. The time ratio is baseline time divided by ComNetX time.

Dataset	Backend	Q	NMI	T (s)	Time ratio
PubMed	Leiden	0.7864/0.7786	0.2017/0.2045	248.6/10.165	24.46
PubMed	DF-Leiden	0.7619/0.7786	0.1946/0.2051	4.442/4.796	0.93
ArxivMath	Leiden	0.8860/0.8799	0.4205/0.4006	5652.7/77.788	72.67
ArxivMath	DF-Leiden	0.8551/0.8792	0.3884/0.3967	134.4/13.517	9.94

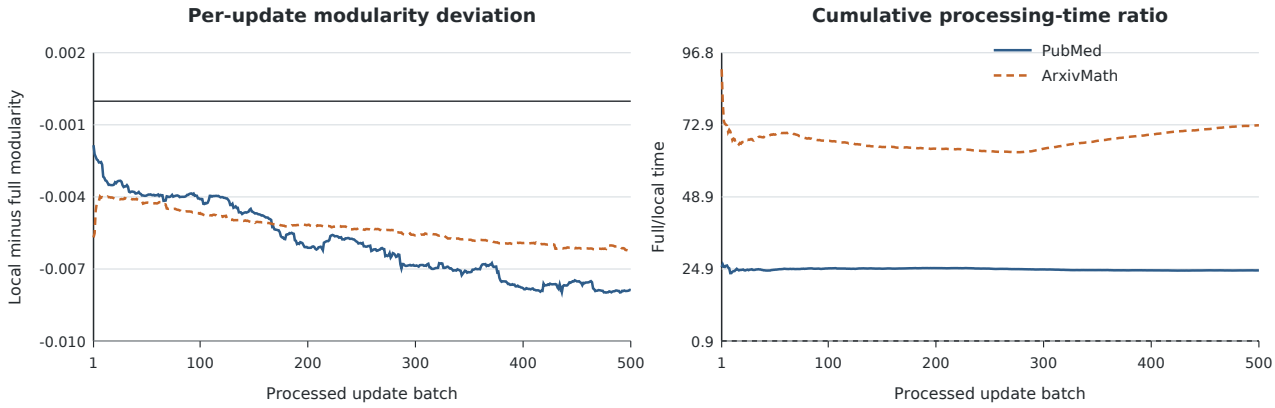


FIGURE 3. Leiden trajectories over 500 updates under 9:500. Left: per-batch modularity difference (ComNetX minus full Leiden), where negative values favor the full partition. Right: cumulative full/ComNetX processing-time ratio, where values above one favor ComNetX. Both curves cover the complete 500-update sequence.

grows from 64.12 to 695.84 as the requested count increases from 5 to 320; the final modularity gap increases from 0.0039 to 0.0134. The result identifies a favorable regime for local maintenance, but the larger ratio also reflects more baseline calls over the same set of stored records. It is not an increase in the amount of graph evolution tested.

PubMed reaches one record per update at 44 updates. Requests for 80, 160, and 320 chunks therefore produce the same 44-update stream. Its Leiden ratios of 45.43–45.57 and identical final quality describe repeated processing of that stream, not three longer trajectories. Across all granularity settings, ComNetX with DF-Leiden has higher final modularity and NMI than the native stream on both graphs; its time ratio is 8.45–10.73 on ArxivMath and below one on PubMed.

3) Increasing the Updated Fraction

The 99:50 control processes approximately 1% of the stored records, between the 0.1% maintenance window and the 10% window of 9:500. Table 14 shows Leiden speedups of 24.16 on PubMed and 60.26 on ArxivMath, with final modularity differences of 0.0074 and 0.0041. DF-Leiden exhibits the same qualitative pattern as at 500 updates: the native stream is slightly faster on PubMed, while ComNetX is faster on ArxivMath and improves both quality measures on both graphs. Since both the initial snapshot and the number of calls change between protocols, these comparisons do not isolate update-window size as a single causal factor.

4) Growth from Singleton Initialization

The ten-bin control tests a substantially larger change in input conditions: the entire graph is processed, and no precomputed

TABLE 13. Fixed-window granularity control, 999:*q*. Here *u* is the actual number of nonempty updates, *b* the stored records per update, ΔQ final ComNetX-minus-baseline modularity, and *S* baseline/ComNetX time. DF-Leiden uses its native dynamic baseline. Each row is one execution per method. The repeated S^{2}CAG comparison at $q = 5, 10$ is given in Table 6.

Dataset	q	u	b	Leiden ΔQ	Leiden S	DF ΔQ	DF S
PubMed	5	5	8–9	-0.0043	5.24	+0.0206	0.32
PubMed	10	10	4–5	-0.0050	31.42	+0.0207	0.71
PubMed	20	20	2–3	-0.0060	39.21	+0.0277	0.61
PubMed	40	40	1–2	-0.0167	44.83	+0.0076	0.49
PubMed	80	44	1	-0.0179	45.57	+0.0077	0.51
PubMed	160	44	1	-0.0179	45.49	+0.0077	0.51
PubMed	320	44	1	-0.0179	45.43	+0.0077	0.51
ArxivMath	5	5	159–160	-0.0039	64.12	+0.0306	8.45
ArxivMath	10	10	79–80	-0.0046	99.58	+0.0309	10.05
ArxivMath	20	20	39–40	-0.0049	149.57	+0.0354	10.34
ArxivMath	40	40	19–20	-0.0065	232.46	+0.0215	10.71
ArxivMath	80	80	9–10	-0.0080	381.84	+0.0193	10.73
ArxivMath	160	160	4–5	-0.0084	556.49	+0.0207	10.42
ArxivMath	320	320	2–3	-0.0134	695.84	+0.0204	9.99

TABLE 14. Intermediate-window control, 99:50: 50 updates collectively covering approximately 1% of the stored record sequence. Values are baseline/ComNetX, with the same baseline and timing conventions as Table 12.

Dataset	Backend	Q	NMI	T (s)	Time ratio
PubMed	Leiden	0.7883/0.7809	0.2049/0.2052	24.831/1.028	24.16
PubMed	DF-Leiden	0.7401/0.7808	0.1875/0.2068	0.447/0.485	0.92
ArxivMath	Leiden	0.8855/0.8813	0.4181/0.4006	571.3/9.480	60.26
ArxivMath	DF-Leiden	0.8377/0.8810	0.3952/0.4002	13.637/1.506	9.05

Leiden partition is supplied. Table 15 reports the five complete paired graphs. ComNetX with Leiden is faster on all five, with ratios of 1.17–1.79 and final modularity differences of 0.0003–0.0080. NMI increases on Patent and PubMed. These smaller speedups show that the method can remain useful during graph growth, while the largest maintenance gains rely on small updates to an established graph and hierarchy.

VII. CODE AVAILABILITY AND LIMITATIONS

The ComNetX implementation, experiment configurations, and archived results are available online.[3] The repository provides dataset preparation instructions. Rerunning S^{2}CAG and native DF-Leiden requires external components not included in the artifact. The exact version of the measured DF-Leiden Python binding was not retained, limiting reproducibility of that baseline.

Four of the six graphs use artificial edge orders, and the window controls reuse the same record sequences. Most configurations have one execution per method. The repeated S^{2}CAG study uses five runs per method and setting to assess runtime repeatability at fixed inputs. Independent temporal windows, edge deletions, arriving vertices, and changing node attributes have not been evaluated.

VIII. CONCLUSION

ComNetX adapts existing community detectors to batched graph updates by selecting complete affected communities and contracting groups defined by a stored hierarchy. The first pass updates the reported partition, and later passes update the coarser levels while preserving nesting. The method reuses the detector implementation, and its analysis separates the cost of selecting the update region from that of partitioning the contracted graph.

The experiments support using ComNetX for maintenance of an established graph: it reduces repeated Leiden processing with small modularity differences over both short and sustained update streams. The advantage is smaller during graph growth. ComNetX also outperforms native DF-Leiden in measured time and final quality on ArxivMath; on PubMed, higher final quality comes with a longer processing time. Repeated paired S^{2}CAG runs also show time savings and higher final Q and NMI under the implemented interface. Their markedly different output group counts make the adapter's feature processing and cluster-count rules essential to interpreting this comparison.

Future work can use update-region and quotient statistics to choose between local and full processing and can extend the evaluation to deletion streams, arriving vertices, and changing node attributes.

APPENDIX A BOUNDARY TERM FOR MODULARITY

To relate modularity on the induced subgraph and on the full graph, assume $A_{ij} \geq 0$, $W = \sum_{i,j} A_{ij} > 0$, and $\gamma \geq 0$. Let $U = U_t^{\ell}$, $W_U = \sum_{i,j \in U} A_{ij} > 0$. Write $\boldsymbol{A}[U] = (A_{ij})_{i,j \in U}$ for the adjacency matrix of the induced subgraph on U, and let Z be a partition of U whose labels are disjoint from the

[3]https://github.com/mpailab/comnetx/tree/ieee-access-r6-20261106/artifacts/ieee-access

TABLE 15. Graph growth under 0:10 from singleton initialization, including the first processed snapshot. Values are full Leiden/ComNetX; each row is one paired execution. The entire stored record sequence is divided into ten batches.

Dataset	Q	NMI	T (s)	Time ratio
Cora	0.8230/0.8150	0.4715/0.4642	0.985/0.640	1.54
ACM	0.8248/0.8232	0.2704/0.2687	0.934/0.801	1.17
CiteSeer	0.8932/0.8860	0.3302/0.3272	0.847/0.473	1.79
Patent	0.8567/0.8564	0.3617/0.3673	2.197/1.351	1.63
PubMed	0.7866/0.7797	0.2014/0.2094	5.652/3.972	1.42

fixed outside partition. Let C_Z combine Z with that retained outside partition. For each block a contained in U, let x_a and y_a be its outgoing and incoming degree within U, and define its boundary degrees by

$$p_a = \sum_{i\in a, j\notin U} A_{ij}, \qquad q_a = \sum_{i\notin U, j\in a} A_{ij}. \tag{24}$$

If $K_{V\setminus U}$ is the full-modularity contribution independent of Z, direct substitution in (5) gives

$$\begin{aligned} D(Z) &= Q_\gamma(\boldsymbol{A}, C_Z) - K_{V\setminus U} - \frac{W_U}{W} Q_\gamma(\boldsymbol{A}[U], Z) \\ &= \frac{\gamma}{W^2}\left[\frac{W - W_U}{W_U}\sum_a x_a y_a \right. \\ &\qquad \left. - \sum_a (x_a q_a + y_a p_a + p_a q_a)\right]. \end{aligned} \tag{25}$$

The internal-edge term is unchanged; the mismatch consists of the global-versus-local null-model normalization and the boundary degrees.

Write $\beta_{\text{out}} = \sum_a p_a$ and $\beta_{\text{in}} = \sum_a q_a$. Nonnegativity gives

$$-B_- \le D(Z) \le B_+, \tag{26}$$

where

$$\begin{aligned} B_+ &= \frac{\gamma}{W^2} W_U (W - W_U), \\ B_- &= \frac{\gamma}{W^2}\left[W_U(\beta_{\text{out}} + \beta_{\text{in}}) + \beta_{\text{out}}\beta_{\text{in}}\right]. \end{aligned} \tag{27}$$

For example, $\sum_a x_a y_a \le W_U^2$, $\sum_a x_a q_a \le W_U \beta_{\text{in}}$, and the analogous bounds for the remaining terms give (26).

For a fixed family $\mathcal{S}$ of partitions that keep the specified atoms intact, define $F(Z) = Q_\gamma(\boldsymbol{A}, C_Z) - K_{V\setminus U}$ and $L(Z) = (W_U/W) Q_\gamma(\boldsymbol{A}[U], Z)$. If Z_U maximizes L and Z_G maximizes F over $\mathcal{S}$, then

$$0 \le F(Z_G) - F(Z_U) \le B_- + B_+. \tag{28}$$

This bound compares exact optima only within that family; it does not bound loss against an unrestricted full-graph optimum or certify the output of a heuristic backend.

APPENDIX B MEMORY USE OF THE CURRENT IMPLEMENTATION

The implementation stores Boolean masks for all fixed update regions and processes levels sequentially. Let $M_{\text{SpMM},\ell}$ and $M_{\mathcal{B},\ell}$ denote sparse-product and backend workspace at level ℓ. Let $M_{\text{feat},\ell}$ denote the additional storage for preparing and aggregating features at that level; it is zero when attributes are not used. The implementation's total additional space is conservatively bounded by

$$\begin{aligned} M_{\text{extra}} = O\Big(Ln + \max\big\{s_t + n, \\ \max_\ell \big[e_\ell + h_\ell + q_\ell + M_{\text{feat},\ell} \\ + M_{\text{SpMM},\ell} + M_{\mathcal{B},\ell}\big]\big\}\Big). \end{aligned} \tag{29}$$

ACKNOWLEDGMENT

The research was carried out using the MSU-270 supercomputer of Lomonosov Moscow State University.

OpenAI Codex (https://openai.com/codex/) was used to assist with language editing and structural revision of the Abstract, Introduction, Related Work, Method, Experimental Design, Results and Discussion, Conclusion, and Appendices; with consistency checks against the archived measurements; and with preparation of analysis and measurement scripts and vector-figure code. The authors reviewed and revised all resulting text, code, figures, citations, and numerical claims and take responsibility for the article's content and conclusions.

REFERENCES

[1] J.-P. Onnela, J. Saramäki, J. Hyvönen, G. Szabó, D. Lazer, K. Kaski, J. Kertész, and A.-L. Barabási, "Structure and tie strengths in mobile communication networks," *Proc. Natl. Acad. Sci. USA*, vol. 104, no. 18, pp. 7332–7336, 2007. [Online]. Available: https://doi.org/10.1073/pnas.0610245104

[2] G. Palla, A.-L. Barabási, and T. Vicsek, "Quantifying social group evolution," *Nature*, vol. 446, no. 7136, pp. 664–667, 2007. [Online]. Available: https://doi.org/10.1038/nature05670

[3] G. Rossetti and R. Cazabet, "Community discovery in dynamic networks: A survey," *ACM Comput. Surv.*, vol. 51, no. 2, pp. 1–37, 2018, art. no. 35. [Online]. Available: https://doi.org/10.1145/3172867

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.: Theory Exp.*, vol. 2008, no. 10, 2008, art. no. P10008. [Online]. Available: https://doi.org/10.1088/1742-5468/2008/10/P10008

[5] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," *Sci. Rep.*, vol. 9, 2019, art. no. 5233. [Online]. Available: https://doi.org/10.1038/s41598-019-41695-z

[6] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 127, pp. 1–21, 2023. [Online]. Available: https://www.jmlr.org/papers/v24/20-998.html

[7] Y. Liu, J. Li, Y. Chen, R. Wu, E. Wang, J. Zhou, S. Tian, S. Shen, X. Fu, C. Meng, W. Wang, and L. Chen, "Revisiting modularity maximization for graph clustering: A contrastive learning perspective," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 1968–1979. [Online]. Available: https://doi.org/10.1145/3637528.3671967

[8] X. Lin, R. Yang, H. Zheng, and X. Ke, "Spectral subspace clustering for attributed graphs," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 789–799. [Online]. Available: https://doi.org/10.1145/3690624.3709207

[9] M. Halappanavar, H. Lu, A. Kalyanaraman, and A. Tumeo, "Scalable static and dynamic community detection using Grappolo," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2017, pp. 1–6. [Online]. Available: https://doi.org/10.1109/HPEC.2017.8091047

[10] D. Zhuang, J. M. Chang, and M. Li, "DynaMo: Dynamic community detection by incrementally maximizing modularity," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 5, pp. 1934–1945, 2021. [Online]. Available: https://doi.org/10.1109/TKDE.2019.2951419

[11] N. Zarayeneh and A. Kalyanaraman, "Delta-Screening: A fast and efficient technique to update communities in dynamic graphs," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1614–1629, 2021. [Online]. Available: https://doi.org/10.1109/TNSE.2021.3067665

[12] S. Sahu, K. Kothapalli, and D. S. Banerjee, "Parallel multicore algorithms for community detection in dynamic graphs," *Int. J. Netw. Comput.*, vol. 15, no. 1, pp. 2–31, 2025. [Online]. Available: https://doi.org/10.15803/ijnc.15.1_2

[13] ——, "Towards dynamic community detection with Leiden algorithm," in *Proc. Supercomputing India (SCI)*. IEEE, 2025, pp. 1–8. [Online]. Available: https://doi.org/10.1109/SCI68648.2025.11333864

[14] M. A. Porter, J.-P. Onnela, and P. J. Mucha, "Communities in networks," *Notices Amer. Math. Soc.*, vol. 56, no. 9, pp. 1082–1097, 1164–1166, 2009. [Online]. Available: https://www.ams.org/notices/200909/rtx090901082p.pdf

[15] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, no. 3–5, pp. 75–174, 2010. [Online]. Available: https://doi.org/10.1016/j.physrep.2009.11.002

[16] F. Radicchi, C. Castellano, F. Cecconi, V. Loreto, and D. Parisi, "Defining and identifying communities in networks," *Proc. Natl. Acad. Sci. USA*, vol. 101, no. 9, pp. 2658–2663, 2004. [Online]. Available: https://doi.org/10.1073/pnas.0400054101

[17] G. W. Flake, S. Lawrence, C. L. Giles, and F. M. Coetzee, "Self-organization and identification of Web communities," *Computer*, vol. 35, no. 3, pp. 66–71, 2002. [Online]. Available: https://doi.org/10.1109/2.989932

[18] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E*, vol. 69, no. 2, 2004, art. no. 026113. [Online]. Available: https://doi.org/10.1103/PhysRevE.69.026113

[19] U. Brandes, D. Delling, M. Gaertler, R. Görke, M. Hoefer, Z. Nikoloski, and D. Wagner, "On modularity clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 20, no. 2, pp. 172–188, 2008. [Online]. Available: https://doi.org/10.1109/TKDE.2007.190689

[20] J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 22, no. 8, pp. 888–905, 2000. [Online]. Available: https://doi.org/10.1109/34.868688

[21] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," *Proc. Natl. Acad. Sci. USA*, vol. 105, no. 4, pp. 1118–1123, 2008. [Online]. Available: https://doi.org/10.1073/pnas.0706851105

[22] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social Networks*, vol. 5, no. 2, pp. 109–137, 1983. [Online]. Available: https://doi.org/10.1016/0378-8733(83)90021-7

[23] H. Lu, M. Halappanavar, and A. Kalyanaraman, "Parallel heuristics for scalable community detection," *Parallel Comput.*, vol. 47, pp. 19–37, 2015. [Online]. Available: https://doi.org/10.1016/j.parco.2015.03.003

[24] S. Sahu, K. Kothapalli, and D. S. Banerjee, "Fast Leiden algorithm for community detection in shared memory setting," in *Proc. 53rd Int. Conf. Parallel Process.*, 2024, pp. 11–20. [Online]. Available: https://doi.org/10.1145/3673038.3673146

[25] V. A. Traag and L. Šubelj, "Large network community detection by fast label propagation," *Sci. Rep.*, vol. 13, 2023, art. no. 2701. [Online]. Available: https://doi.org/10.1038/s41598-023-29610-z

[26] Y. Ma, Y. Zhao, J. Wang, M. Liu, W. Shen, and Y. Ma, "Modularity-based incremental label propagation algorithm for community detection," *Appl. Sci.*, vol. 10, no. 12, 2020, art. no. 4060. [Online]. Available: https://doi.org/10.3390/app10124060

[27] H. C. Rustamaji, W. A. Kusuma, S. Nurdiati, and I. Batubara, "Community detection with greedy modularity disassembly strategy," *Sci. Rep.*, vol. 14, 2024, art. no. 4694. [Online]. Available: https://doi.org/10.1038/s41598-024-55190-7

[28] S. Taibi, L. Toumi, and S. Bouamama, "Complex network community discovery using fast local move iterated greedy algorithm," *J. Supercomput.*, vol. 81, no. 1, 2025, art. no. 182. [Online]. Available: https://doi.org/10.1007/s11227-024-06614-8

[29] B. H. Good, Y.-A. de Montjoye, and A. Clauset, "Performance of modularity maximization in practical contexts," *Phys. Rev. E*, vol. 81, no. 4, 2010, art. no. 046106. [Online]. Available: https://doi.org/10.1103/PhysRevE.81.046106

[30] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *Phys. Rep.*, vol. 659, pp. 1–44, 2016. [Online]. Available: https://doi.org/10.1016/j.physrep.2016.09.002

[31] Z. Yang, R. Algesheimer, and C. J. Tessone, "A comparative analysis of community detection algorithms on artificial networks," *Sci. Rep.*, vol. 6, 2016, art. no. 30750. [Online]. Available: https://doi.org/10.1038/srep30750

[32] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," *Knowl. Inf. Syst.*, vol. 42, no. 1, pp. 181–213, 2015. [Online]. Available: https://doi.org/10.1007/s10115-013-0693-z

[33] L. Peel, D. B. Larremore, and A. Clauset, "The ground truth about metadata and community detection in networks," *Sci. Adv.*, vol. 3, no. 5, 2017, art. no. e1602548. [Online]. Available: https://doi.org/10.1126/sciadv.1602548

[34] L. Danon, A. Díaz-Guilera, J. Duch, and A. Arenas, "Comparing community structure identification," *J. Stat. Mech.: Theory Exp.*, vol. 2005, no. 9, 2005, art. no. P09008. [Online]. Available: https://doi.org/10.1088/1742-5468/2005/09/P09008

[35] W. H. Chong and L.-N. Teow, "An incremental batch technique for community detection," in *Proc. 16th Int. Conf. Inf. Fusion*. IEEE, 2013, pp. 750–757. [Online]. Available: https://ieeexplore.ieee.org/document/6641069/

[36] J. Xie, M. Chen, and B. K. Szymanski, "LabelRankT: Incremental community detection in dynamic networks via label propagation," in *Proc. 4th Workshop Dyn. Netw. Manage. Mining*, 2013, pp. 25–32. [Online]. Available: https://doi.org/10.1145/2489247.2489249

[37] J. Han, W. Li, L. Zhao, Z. Su, Y. Zou, and W. Deng, "Community detection in dynamic networks via adaptive label propagation," *PLOS ONE*, vol. 12, no. 11, 2017, art. no. e0188655. [Online]. Available: https://doi.org/10.1371/journal.pone.0188655

[38] J. Shang, L. Liu, F. Xie, Z. Chen, J. Miao, X. Fang, and C. Wu, "A real-time detecting algorithm for tracking community structure of dynamic networks," arXiv:1407.2683 [cs.SI], Jul. 2014. [Online]. Available: https://arxiv.org/abs/1407.2683

[39] M. Seifikar, S. Farzi, and M. Barati, "C-Blondel: An efficient Louvain-based dynamic community detection algorithm," *IEEE Trans. Comput. Social Syst.*, vol. 7, no. 2, pp. 308–318, 2020. [Online]. Available: https://doi.org/10.1109/TCSS.2020.2964197

[40] S. Sahu, "DF Louvain: Fast incrementally expanding approach for community detection on dynamic graphs," arXiv:2404.19634 [cs.DC], Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.19634

[41] N. S. Sattar, A. Buluç, K. Z. Ibrahim, and S. Arifuzzaman, "Exploring temporal community evolution: Algorithmic approaches and parallel optimization for dynamic community detection," *Appl. Netw. Sci.*, vol. 8, 2023, art. no. 64. [Online]. Available: https://doi.org/10.1007/s41109-023-00592-1

[42] N. S. Sattar, K. Z. Ibrahim, A. Buluç, and S. Arifuzzaman, "DyG-DPCD: A distributed parallel community detection algorithm for large-scale dynamic graphs," *Int. J. Parallel Program.*, vol. 53, no. 1, 2025, art. no. 4. [Online]. Available: https://doi.org/10.1007/s10766-024-00780-1

[43] K. Hanauer, M. Henzinger, and C. Schulz, "Recent advances in fully dynamic graph algorithms: A quick reference guide," *ACM J. Exp. Algorithmics*, vol. 27, no. 1, pp. 1.11:1–1.11:45, 2022, art. no. 1.11. [Online]. Available: https://doi.org/10.1145/3555806

[44] S. Sahu, "Efficient tracking of communities on evolving graphs with Leiden algorithm," in *Proc. 35th Int. Symp. High-Perform. Parallel Distrib. Comput.* ACM, 2026, pp. 498–508. [Online]. Available: https://doi.org/10.1145/3806645.3807601

[45] G. Bokov, A. Konovalov, A. Uporova, S. Moiseev, I. Safonov, and A. Radionov, "LD-Leiden: Local parallel community detection in large dynamic networks," arXiv:2502.18497v2 [cs.SI], Jul. 2026. [Online]. Available: https://arxiv.org/abs/2502.18497v2

[46] C. Lin, Y. Xie, Y. Fang, Y. Hu, Y. Hu, and C. Chen, "Maintaining Leiden communities in large dynamic graphs," arXiv:2601.08554v5 [cs.SI], Jun. 2026. [Online]. Available: https://arxiv.org/abs/2601.08554v5

[47] L. Cai, J. Zhou, and D. Wang, "Improving temporal smoothness and snapshot quality in dynamic network community discovery using NOME algorithm," *PeerJ Comput. Sci.*, vol. 9, 2023, art. no. e1477. [Online]. Available: https://doi.org/10.7717/peerj-cs.1477

[48] K. S. Xu and A. O. Hero, III, "Dynamic stochastic blockmodels for time-evolving social networks," *IEEE J. Sel. Topics Signal Process.*, vol. 8, no. 4, pp. 552–562, 2014. [Online]. Available: https://doi.org/10.1109/JSTSP.2014.2310294

[49] A. Bovet, J.-C. Delvenne, and R. Lambiotte, "Flow stability for dynamic community detection," *Sci. Adv.*, vol. 8, no. 19, 2022, art. no. eabj3063. [Online]. Available: https://doi.org/10.1126/sciadv.abj3063

[50] Z. Sun, Y. Sun, X. Chang, F. Wang, Z. Pan, G. Wang, and J. Liu, "Dynamic community detection based on the Matthew effect," *Physica A*, vol. 597, 2022, art. no. 127315. [Online]. Available: https://doi.org/10.1016/j.physa.2022.127315

[51] M. Mazza, G. Cola, and M. Tesconi, "Modularity-based approach for tracking communities in dynamic social networks," *Knowl.-Based Syst.*, vol. 281, 2023, art. no. 111067. [Online]. Available: https://doi.org/10.1016/j.knosys.2023.111067

[52] V. Brabant, A. Bonifati, and R. Cazabet, "Discovering communities in continuous-time temporal networks by optimizing L-modularity," in *Proc. IEEE Int. Conf. Data Mining*. IEEE, 2025, pp. 1065–1074. [Online]. Available: https://doi.org/10.1109/ICDM65498.2025.00115

[53] N. Djurdjevac Conrad, E. Tonello, J. Zonker, and H. Siebert, "Detection of dynamic communities in temporal networks with sparse data," *Appl. Netw. Sci.*, vol. 10, no. 1, 2025, art. no. 1. [Online]. Available: https://doi.org/10.1007/s41109-024-00687-3

[54] F. Blašković, T. O. F. Conrad, S. Klus, and N. Djurdjevac Conrad, "Random walk based snapshot clustering for detecting community dynamics in temporal networks," *Sci. Rep.*, vol. 15, no. 1, 2025, art. no. 24414. [Online]. Available: https://doi.org/10.1038/s41598-025-09340-0

[55] D. J. DiTursi, G. Ghosh, and P. Bogdanov, "Local community detection in dynamic networks," in *Proc. IEEE Int. Conf. Data Mining*, 2017, pp. 847–852. [Online]. Available: https://doi.org/10.1109/ICDM.2017.103

[56] K. Christopoulos, G. Baltsou, and K. Tsichlas, "Local community detection in graph streams with anchors," *Information*, vol. 14, no. 6, 2023, art. no. 332. [Online]. Available: https://doi.org/10.3390/info14060332

[57] J. Liu, Y. Shao, and S. Su, "Multiple local community detection via high-quality seed identification over both static and dynamic networks," *Data Sci. Eng.*, vol. 6, no. 3, pp. 249–264, 2021. [Online]. Available: https://doi.org/10.1007/s41019-021-00160-6

[58] S. Boudebza, R. Cazabet, F. Azouaou, and O. Nouali, "OLCPM: An online framework for detecting overlapping communities in dynamic social networks," *Comput. Commun.*, vol. 123, pp. 36–51, 2018. [Online]. Available: https://doi.org/10.1016/j.comcom.2018.04.003

[59] P. Chunaev, "Community detection in node-attributed social networks: A survey," *Comput. Sci. Rev.*, vol. 37, 2020, art. no. 100286. [Online]. Available: https://doi.org/10.1016/j.cosrev.2020.100286

[60] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, 2009. [Online]. Available: https://doi.org/10.1109/TNN.2008.2005605

[61] Y. Zhang, Y. Xiong, Y. Ye, T. Liu, W. Wang, Y. Zhu, and P. S. Yu, "SEAL: Learning heuristics for community detection with generative adversarial networks," in *Proc. 26th ACM SIGKDD Conf. Knowl. Discovery Data Mining*. ACM, 2020, pp. 1103–1113. [Online]. Available: https://doi.org/10.1145/3394486.3403154

[62] T. Zhang, Y. Xiong, J. Zhang, Y. Zhang, Y. Jiao, and Y. Zhu, "CommDGI: Community detection oriented deep graph infomax," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.* ACM, 2020, pp. 1843–1852. [Online]. Available: https://doi.org/10.1145/3340531.3412042

[63] A. Bhowmick, M. Kosan, Z. Huang, A. Singh, and S. Medya, "DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 10, 2024, pp. 11 069–11 077. [Online]. Available: https://doi.org/10.1609/aaai.v38i10.28983

[64] Devesh Kumar V V, S Sreesankar, Krishnpriya Dinesan, Pranav B Nair, and L R Deepthi, "Analyzing GNN models for community detection using graph embeddings: A comparative study," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol.* IEEE, 2024, pp. 1–8. [Online]. Available: https://doi.org/10.1109/ICCCNT61001.2024.10725530

[65] Y. Liu, J. Xia, B. Wu, S. Zhou, X. Yang, K. Liang, C. Fan, Y. Zhuang, G. Yu, S. Z. Li, X. Liu, and K. He, "A survey of deep graph clustering: Taxonomy, challenge, application, and open resource," *IEEE Trans. Knowl. Data Eng.*, vol. 38, no. 6, pp. 3365–3384, 2026. [Online]. Available: https://doi.org/10.1109/TKDE.2026.3669747

[66] C. Liu, Y. Yang, Y. Ding, H. Lu, W. Lin, Z. Wu, and W. Bi, "DAG: Deep adaptive and generative K-free community detection on attributed graphs," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*. ACM, 2024, pp. 5454–5465. [Online]. Available: https://doi.org/10.1145/3637528.3671615

[67] J. Zhang, H. Peng, L. Sun, G. Wu, C. Liu, and Z. Yu, "Unsupervised graph clustering with deep structural entropy," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining V.2*, vol. 2. ACM, 2025, pp. 3752–3763. [Online]. Available: https://doi.org/10.1145/3711896.3737173

[68] C. Gkartzios, E. Pitoura, and P. Tsaparas, "Modularity-fair deep community detection," in *Proc. IEEE Int. Conf. Data Mining*. IEEE, 2025, pp. 287–296. [Online]. Available: https://doi.org/10.1109/ICDM65498.2025.00036

[69] M. Qin, C. Zhang, Y. Gao, W. Zhang, and D.-Y. Yeung, "Pre-train and refine: Towards higher efficiency in K-agnostic community detection without quality degradation," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*. ACM, 2024, pp. 2467–2478. [Online]. Available: https://doi.org/10.1145/3637528.3671686

[70] M. Qin, C. Zhang, Y. Gao, Y. Ding, W. Jiang, W. Zhang, W. Han, and B. Bai, "Towards faster graph partitioning via pre-training and inductive inference," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2024, pp. 1–7. [Online]. Available: https://doi.org/10.1109/HPEC62836.2024.10938459

[71] D. Kong, A. Zhang, and Y. Li, "Learning persistent community structures in dynamic networks via topological data analysis," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 8, 2024, pp. 8617–8626. [Online]. Available: https://doi.org/10.1609/aaai.v38i8.28706

[72] S. Ji, X. Lu, M. Liu, L. Sun, C. Liu, B. Du, and H. Xiong, "Community-based dynamic graph learning for popularity prediction," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*. ACM, 2023, pp. 930–940. [Online]. Available: https://doi.org/10.1145/3580305.3599281

[73] A. Arenas, J. Duch, A. Fernández, and S. Gómez, "Size reduction of complex networks preserving modularity," *New J. Phys.*, vol. 9, no. 6, 2007, art. no. 176. [Online]. Available: https://doi.org/10.1088/1367-2630/9/6/176

[74] M. Liu, K. Liang, S. Wang, X. Hu, S. Zhou, and X. Liu, "Deep temporal graph clustering: A comprehensive benchmark and datasets," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 12, pp. 11 561–11 578, 2025. [Online]. Available: https://doi.org/10.1109/TPAMI.2025.3596609

[75] M. Liu, K. Liang, Y. Liu, S. Wang, S. Zhou, and X. Liu, "arXiv4TGC: Large-scale datasets for temporal graph clustering," arXiv:2306.04962 [cs.AI], Jun. 2023. [Online]. Available: https://arxiv.org/abs/2306.04962

[76] Z. Yang, W. W. Cohen, and R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," in *Proc. 33rd Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 48. PMLR, 2016, pp. 40–48. [Online]. Available: https://proceedings.mlr.press/v48/yanga16.html

GRIGORIY BOKOV received the Candidate of Physical and Mathematical Sciences degree in 2014. He is currently the Director of the AI Research Center, Lomonosov Moscow State University, Moscow, Russia, and an Associate Professor and the Head of the Laboratory of Mathematical Problems of Artificial Intelligence at the Faculty of Mechanics and Mathematics. His research interests include logical calculi, functional systems, automata, machine-learning algorithms, optimization, and large dynamic graphs.

ALEKSANDR KONOVALOV received the Specialist degree from the Faculty of Mechanics and Mathematics, Lomonosov Moscow State University, Moscow, Russia, in 2013, and the Candidate of Physical and Mathematical Sciences degree in 2018. He is currently a Junior Researcher with Lomonosov Moscow State University. His research interests include constructive mathematical logic, graph neural networks, and dynamic community detection.

ANNA UPOROVA is currently pursuing a Specialist degree in the Faculty of Mechanics and Mathematics, Lomonosov Moscow State University, Moscow, Russia. Her research interests include dynamic graphs, community detection, and graph neural networks.

ALEXANDER DROBYSHEV is currently a postgraduate student with the Chair of Mathematical Theory of Intelligent Systems, Faculty of Mechanics and Mathematics, Lomonosov Moscow State University, Moscow, Russia. His research interests include Boolean-network dynamics, limit cycles, computational complexity, and dynamic graph algorithms.

IAROSLAV EGOROV is affiliated with the AI Research Center, Lomonosov Moscow State University, Moscow, Russia. His recent work includes scalable dynamic community detection.